Reviewer Pack — Minimal Number of Automorphism Group Orbits and Communicatio…

Entry c0d872afa484 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 16:19:06 UTC

Minimal Number of Automorphism Group Orbits and Communication Complexity Rank Variance Ratio

Entry ID: `c0d872afa484` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 16:19:06 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Automorphism Groups) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any given n-vertex graph G, the ratio between the number of orbits under the automorphism group action |Aut(G)| and the rank of the adjacency matrix of G, |Adj(G)|, satisfies |Aut(G)|/|Adj(G)| ≥ α(n), where α(n) is a function that grows at least linearly with n.

Rationale (proposer's reasoning):

> The conjecture links the algebraic structure of automorphism groups with the complexity of communication tasks on graphs. Automorphism groups can capture symmetries in a graph, and their orbits could be related to the number of distinct communication protocols required for certain tasks. If this ratio is indeed linear or higher, it would suggest that the symmetry of a graph is a resource-intensive property in terms of communication.

Taxonomy category: `GROUP_COMMUNICATION_COMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `74d840736a800b7d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The ratio between the number of automorphism group orbits and the adjacency matrix rank |Aut(G)|/|Adj(G)| for a graph G with n vertices meets the condition that |Aut(G)|/|Adj(G)| ≥ α(n) where α(n) is linearly related to n.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "automorphism group orbits" AND "communication complexity rank variance" - "adjacency matrix rank" AND automorphism group action" - "linear growth function" IN "automorphism group" AND "matrix rank"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n, m):
        if n <= 1 or m < n - 1:
            return None
        graph = [[0] * n for _ in range(n)]
        edges = set()
        while len(edges) < m:
            u, v = random.sample(range(n), 2)
            if u != v and (u, v) not in edges and (v, u) not in edges:
                graph[u][v] = 1
                graph[v][u] = 1
                edges.add((u, v))
        return graph

    def is_connected(graph):
        n = len(graph)
        visited = [False] * n
        stack = [0]
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                for v in range(n):
                    if graph[u][v] == 1 and not visited[v]:
                        stack.append(v)
        return all(visited)

    def find_orbits(graph):
```

```
    n = len(graph)
    orbits = []
    visited = [False] * n
    for i in range(n):
        if not visited[i]:
            orbit = []
            queue = [i]
            while queue:
                u = queue.pop()
                if not visited[u]:
                    visited[u] = True
                    orbit.append(u)
                    for v in range(n):
                        if graph[u][v] == 1 and not visited[v]:
                            queue.append(v)
            orbits.append(orbit)
    return orbits

def matrix_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(min(m, n)):
        pivot = None
        for j in range(i, m):
            if matrix[j][i] != 0:
                pivot = j
                break
        if pivot is not None:
            rank += 1
            for k in range(n):
                matrix[i][k], matrix[pivot][k] = matrix[pivot][k], matrix[i][k]
            for j in range(m):
                if j != i:
                    factor = -matrix[j][i] / matrix[i][i]
                    for k in range(n):
                        matrix[j][k] += factor * matrix[i][k]
    return rank

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):
        graph = generate_graph(n, int(3 * n / 4))
        if not is_connected(graph):
            continue
        orbits = find_orbits(graph)
        rank = matrix_rank(graph)
        if rank == 0:
            continue
        metric_value = len(orbits) / rank
        total_metric_value += metric_value
```

```
            instances_tested += 1
            n_max = max(n_max, n)

            if conjecture_holds and metric_value < n:
                conjecture_holds = False
                counterexample = f"Orbits: {len(orbits)}, Rank: {rank}, Ratio: {metric_value}"

    return {
        "metric_name": "Orbit Width Ratio",
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_965a71d7.py", line 126, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_965a71d7.py", line 96, in run_trial
    if not is_connected(graph):
           ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_965a71d7.py", line 35, in is_connected
    n = len(graph)
        ^^^^^^^^^^
```

```
TypeError: object of type 'NoneType' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture. | next: Review and debug the test code to ensure it can run successfully and produce results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14714
2	propose	ollama_remote	glm4:latest	0	0	12754
3	propose	ollama_remote	glm4:latest	0	0	16038
4	preregistration	ollama_remote	glm4:latest	0	0	10538
5	novelty	ollama_remote	glm4:latest	0	0	8465
6	novelty	ollama_remote	glm4:latest	0	0	9151
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19520
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20180
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15040
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14289
11	judge	ollama_remote	glm4:latest	0	0	17277

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 157964 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/c0d872afa484.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c0d872afa484.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c0d872afa484.tar.gz` (if generated)